

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчёт

По лабораторной работе №4

По дисциплине «Логика и основа алгоритмизации в информационных системах».

На тему «Бинарное дерево поиска»

Выполнили студенты группы 25ВВВЗ:

Ратьков Д. И.

Градсков А.Д

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Деев М.В

Пенза 2026

ЛАБОРАТОРНАЯ РАБОТА №4

Бинарное дерево поиска

Цель работы: Изучение структуры данных "бинарное дерево" и реализация базовых операций работы с ним: поиск элементов, подсчет вхождений заданного значения, а также модификация алгоритма добавления для обеспечения уникальности элементов. Анализ вычислительной сложности операций поиска в бинарном дереве.

Практическая часть

Задание:

1. Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.
2. Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.
3. Изменить функцию добавления элементов для исключения добавления одинаковых символов.
4. Оценить сложность процедуры поиска по значению в бинарном дереве.

Реализация задания:

Результат выполнения программы:

Задание 1:

```
C:\Users\grads\OneDrive\Desl  ×  +  ▾  
Разрешить повторяющиеся элементы в дереве?  
1 – да (обычное добавление), 0 – нет (без дублей, задание 3*): 1  
  
Построение дерева  
-1 – окончание построения дерева  
Введите число: 3  
Введите число: 2  
Введите число: 3  
Введите число: 7  
Введите число: 6  
Введите число: 9  
Введите число: 1  
Введите число: -1  
Построение дерева окончено  
  
Полученное дерево (повернуто на 90 градусов, корень слева):  
1  
  2  
    3  
3  
  6  
    7  
      9  
  
Поиск элемента  
Введите число для поиска: 2  
Элемент 2 найден в дереве  
  
Подсчёт числа вхождений  
Введите число для подсчёта вхождений: |
```

Задание 2:

```
Консоль отладки Microsoft Vi  X  +  v

Разрешить повторяющиеся элементы в дереве?
1 - да (обычное добавление), 0 - нет (без дублей, задание 3*): 1

Построение дерева
-1 - окончание построения дерева
Введите число: 3
Введите число: 2
Введите число: 3
Введите число: 7
Введите число: 6
Введите число: 9
Введите число: 1
Введите число: -1
Построение дерева окончено

Полученное дерево (повернуто на 90 градусов, корень слева):
      1
     / \
    2   3
   / \ / \
  3  6 7  9

Поиск элемента
Введите число для поиска: 2
Элемент 2 найден в дереве

Подсчёт числа вхождений
Введите число для подсчёта вхождений: 3
Элемент 3 встречается в дереве 2 раз(а)

C:\Users\grads\OneDrive\Desktop\Университет\ЛиОАВИЗ\4\4\x64\Debug\4.exe (процесс 23
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сер
новке отладки".
Нажмите любую клавишу, чтобы закрыть это окно...
```

Задание 3:

```
C:\Users\grads\OneDrive\Desl  X  +  v

Разрешить повторяющиеся элементы в дереве?
1 - да (обычное добавление), 0 - нет (без дублей, задание 3*): 0

Построение дерева
-1 - окончание построения дерева
Введите число: 1
Введите число: 1
Элемент 1 уже есть в дереве, добавление пропущено
Введите число: 3
Введите число: 5
Введите число: 2
Введите число: 9
Введите число: 6
Введите число: 8
Введите число: 3
Элемент 3 уже есть в дереве, добавление пропущено
Введите число: -1
Построение дерева окончено

Полученное дерево (повернуто на 90 градусов, корень слева):
1
  2
   3
    5
     6
      8
       9

Поиск элемента
Введите число для поиска: |
```

Задание 4. Оценка сложности:

Сложность поиска в бинарном дереве определяется числом сравнений, которое требуется сделать, чтобы прийти от корня до искомого узла (или до NULL, если элемента нет), то есть высотой дерева h на пути поиска:

- В сбалансированном (или построенном из случайных, «перемешанных» данных) дереве высота дерева составляет $h \approx \log_2(n)$, где n — число узлов, поэтому поиск выполняется за $O(\log n)$.
- В худшем случае — если элементы добавлялись в уже отсортированном (возрастающем или убывающем) порядке — дерево вырождается в подобие линейного списка, где каждый узел имеет только одного потомка. Высота такого дерева равна n , и сложность поиска ухудшается до $O(n)$.
- В среднем случае, для дерева, построенного из случайной перестановки n различных значений, математическое ожидание высоты дерева составляет $O(\log n)$, поэтому средняя сложность поиска также $O(\log n)$.

Таким образом, реализованная функция `SearchTree` в лучшем и среднем случае работает за $O(\log n)$, а в худшем случае (вырожденное дерево) — за $O(n)$. Запрет на добавление дублирующихся элементов (задание 3) не меняет асимптотику поиска — он лишь исключает лишние обходы к узлам с уже встречавшимся значением.

Ссылка на репозиторий: <https://github.com/AndreyPayPuv/4lab.git>

Вывод: В ходе выполнения лабораторной работы изучили структуры данных "бинарное дерево" и реализацию базовых операций работы с ним: поиск элементов, подсчет вхождений заданного значения, а также модификация алгоритма добавления для обеспечения уникальности элементов. Анализировали вычислительную сложность операций поиска в бинарном дереве.